

SIMATS ENGINEERING

ASSESSMENT TOOL 2 — MODEL ANSWERS

Course: Internet Programming (CSA43) | CO-1: HTML/XHTML, XML, Internet Protocols & HTTP
Assessment Tool: Data Interpretation

Question 1: HTTP Response Analysis

Given header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

1. Identify any incorrect or invalid fields in the header.

- Content-Length: -150 is invalid — the Content-Length header must be a non-negative integer representing the number of bytes in the response body. A negative value has no meaning and violates the HTTP specification (RFC 7230/9110).
- All other fields are syntactically valid: the status line (HTTP/1.1 200 OK), Date, Content-Type: text/html, and Connection: keep-alive are correctly formatted.

2. Analyze the issue with Content-Length and explain its impact.

Content-Length tells the client exactly how many bytes of body data to expect after the headers, so the browser knows when the message ends (especially important for persistent/keep-alive connections where multiple responses share one TCP connection). A negative value is nonsensical:

- The browser/HTTP client cannot determine a valid body size, so it may reject the response outright, throw a protocol/parsing error, or behave unpredictably.
- On a keep-alive connection, an invalid length makes it impossible for the client to know where this response ends and the next one begins, which can corrupt or hang the connection (response smuggling / framing errors).
- This typically indicates a server-side bug — e.g., an integer underflow/miscalculation when computing the body size — rather than an intentional value.

3. Determine whether the status code matches the response correctness.

The status code 200 OK indicates the request succeeded and a valid resource is being returned. However, this is inconsistent with the malformed Content-Length field: a response cannot simultaneously claim complete success (200 OK) while providing headers that make it impossible to correctly parse the body. In practice, the status line alone is well-formed, but the response as a whole is **not valid** because of the corrupted Content-Length — the header block does not correctly describe the body that is claimed to exist.

4. Suggest corrections for the identified errors.

- Set Content-Length to the correct, non-negative byte count of the actual response body, e.g., Content-Length: 150 (or whatever the true size is).
- Alternatively, if the exact size cannot be known in advance (e.g., streamed/dynamically generated content), use Transfer-Encoding: chunked instead of Content-Length.
- Fix the server-side logic that calculates the content length (check for subtraction errors, buffer miscalculations, or encoding mismatches).

- Corrected header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive
```

Question 2: DOM Structure Analysis

Given code:

```
<html>
<head>
<title>Test Page</title>
</head>
<body>
<div><p>Hello World
</div>
</body>
</html>
```

1. Identify missing or improperly closed elements.

- `<p>Hello World` is opened but never explicitly closed with `</p>` before `</div>` appears.
- There is no `<!DOCTYPE html>` declaration at the top of the document.
- The `<html>` tag has no `lang` attribute, and `<head>` has no `<meta charset>` declaration.
- The `<div>` itself is properly closed, but because its child `<p>` is not, the parser has to decide how to close `<p>` automatically.

2. Analyze how the browser will interpret this structure.

Per the HTML5 parsing algorithm, browsers apply automatic error correction for unclosed tags. Since `<p>` is a block-level element, the browser's parser will implicitly close it as soon as it encounters the next element that cannot be nested inside a `<p>` (here, the closing `</div>` tag). Effectively, the browser constructs the DOM as:

```
html
├── head
│   └── title: "Test Page"
├── body
│   └── div
│       └── p
│           └── "Hello World" (p auto-closed here, before </div>)
```

So modern, standards-compliant browsers will silently self-close the `<p>` right before `</div>`, producing a technically working (though invalid) DOM. Because there is no DOCTYPE, however, the page may render in quirks mode, which can alter box-model and layout calculations.

3. Determine the impact on rendering and layout.

- In most cases the visible text "Hello World" still renders correctly, because the browser's error-recovery silently fixes the missing tag — so the immediate visual impact is minimal.
- However, missing the DOCTYPE can trigger quirks mode, changing how margins, padding, and box-sizing are computed, leading to unpredictable spacing/layout differences across browsers.
- Because the DOM the browser builds differs from the DOM implied by the source (an inferred closing tag was inserted for you), CSS selectors, JavaScript DOM traversal (e.g., `div.children`), and automated tools/validators can behave unexpectedly or fail validation.
- Poorly closed tags reduce maintainability and can cause cascading errors in larger documents, where the browser's auto-correction guesses incorrectly.

4. Suggest corrections to make the DOM valid.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Test Page</title>
</head>
```

```
<body>
  <div>
    <p>Hello World</p>
  </div>
</body>
</html>
```

Adding the DOCTYPE forces standards mode, the meta charset ensures correct text encoding, and explicitly closing </p> before </div> removes any ambiguity about nesting so the DOM matches the author's intent exactly.

Question 3: GET vs POST in Login System

Given comparison:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

1. Interpret the differences in data transmission between GET and POST.

- GET appends form data as a query string directly onto the request URL (e.g., /login?user=admin&pass=1234), so the data travels as part of the URL itself.
- POST places the form data inside the body of the HTTP request, separate from the URL — the request line simply reads POST /login HTTP/1.1, with the credentials sent afterward as body content.
- Because URLs are logged, cached, bookmarked, and displayed in the browser's address bar, GET data is far more exposed than POST data, which stays out of the visible URL.

2. Analyze which method is more suitable for login and why.

POST is more suitable for a login system. Login is a state-changing, sensitive operation, and POST requests are not cached, not stored in browser history, and not automatically re-sent when a page is refreshed/bookmarked (unlike GET). Keeping the username and password in the request body — rather than the URL — significantly reduces accidental exposure. GET, by contrast, is intended for safe, idempotent operations like fetching a resource, and was never designed to carry confidential data.

3. Identify security risks in using GET for authentication.

- Credentials appear directly in the URL, so they get stored in browser history and can be seen by anyone with access to the device.
- URLs are commonly logged by web servers, proxies, and load balancers in plaintext access logs — meaning passwords could sit in log files indefinitely.
- URLs can be leaked via the HTTP Referer header when the logged-in page links to external resources, exposing credentials to third-party sites.
- URLs are easy to share, bookmark, or screenshot accidentally, and GET requests may be pre-fetched or cached by browsers/CDNs, further increasing exposure.

4. Suggest best practices for secure login implementation.

- Always use method="POST" for login forms, submitted only over HTTPS (TLS) so the body itself is encrypted in transit.
- Hash and salt passwords server-side (e.g., bcrypt/Argon2); never transmit or store plaintext passwords.
- Use secure, HttpOnly, SameSite cookies for session tokens after successful login, rather than passing credentials on every request.
- Implement CSRF protection, rate limiting/account lockout after repeated failed attempts, and consider multi-factor authentication (MFA).
- Apply input validation on both client and server sides, and avoid exposing detailed error messages (e.g., "invalid password" vs. "invalid username") that could aid attackers.

Question 4: HTML Code Inspection

Given code:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

1. Identify missing closing tags.

- `<h1>Welcome` has no closing `</h1>` tag.
- `<p>This is a paragraph` has no closing `</p>` tag.
- The `<html>` element has no `lang` attribute, and `<head>` has no `<meta charset>` declaration (not a closing-tag issue, but still non-compliant).
- Note: `<img>` is a void/self-closing element by definition, so it correctly requires no closing tag — this is not an error.

2. Analyze issues with the `<img>` tag.

- Missing `alt` attribute — every `<img>` should include descriptive `alt` text for accessibility (screen readers) and for graceful degradation if the image fails to load.
- No `width/height` attributes are specified, which can cause layout shift (content "jumping") while the image loads, hurting page stability and Core Web Vitals.
- The `src` path ("image.jpg") is relative with no confirmation of correct location/case-sensitivity — a common source of broken images, especially on case-sensitive servers (Linux) versus case-insensitive local testing (Windows).

3. Determine how browsers will handle these errors.

Modern browsers apply the HTML5 error-recovery parsing algorithm:

- The unclosed `<h1>` will be auto-closed by the parser as soon as the next block-level element (`<p>`) begins, since a heading cannot contain a paragraph.
- Similarly, the unclosed `<p>` will be auto-closed once the parser reaches an element that a `<p>` cannot contain (here, effectively at the `<img>` or the closing `</body>`).
- Because `<img>` is a void element, its lack of a closing tag causes no issue at all — browsers expect it to self-close.
- If the `alt` attribute is missing and the image fails to load, the browser will show a broken-image icon with no descriptive fallback text, degrading the user experience and accessibility.

4. Suggest a corrected version of the code.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Sample</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph</p>
```

```

</body>
</html>
```

Question 5: Browser-Server Communication Flow

Given sequence: 1) User enters a URL → 2) DNS lookup → 3) HTTP request sent → 4) Server processes request → 5) Response returned → 6) Browser renders page.

1. Interpret each step in the communication flow.

- Step 1 — User enters a URL: the browser parses the URL into its components (protocol, host, path, query string).
- Step 2 — DNS lookup: the browser resolves the human-readable hostname (e.g., `www.example.com`) into an IP address by querying DNS resolvers/servers.
- Step 3 — HTTP request sent: the browser opens a TCP connection (and TLS handshake if HTTPS) to that IP address and sends an HTTP request (method, headers, and body if applicable).
- Step 4 — Server processes request: the web server (and any backend/application logic, database queries, etc.) handles the request and prepares a response.
- Step 5 — Response returned: the server sends back an HTTP response containing a status code, headers, and the response body (HTML, JSON, images, etc.).
- Step 6 — Browser renders page: the browser parses the HTML into the DOM, fetches linked resources (CSS, JS, images), builds the render tree, and paints the final page on screen.

2. Identify where delays can occur.

- DNS lookup delay — if the hostname is not cached, resolving it through multiple DNS servers can add latency.
- Connection setup delay — TCP handshake and, for HTTPS, the additional TLS handshake add round trips before any data is exchanged.
- Server processing delay — slow database queries, heavy backend computation, or server overload can delay step 4 significantly.
- Network/transmission delay — physical distance, bandwidth limits, and congestion affect how quickly request/response data travels.
- Rendering delay — large HTML/CSS/JS payloads, render-blocking scripts/stylesheets, and unoptimized images can slow down step 6 even after the response has fully arrived.

3. Analyze the role of DNS in the process.

DNS (Domain Name System) acts as the internet's "phone book," translating a human-friendly domain name into the numeric IP address that is actually needed to route network packets. Without this step, the browser would have no way to know which server to connect to. DNS resolution typically involves checking the browser cache, OS cache, router, ISP resolver, and finally the authoritative name servers if needed — a chain that, on a cache miss, can noticeably add to overall page-load time. This is why DNS caching and prefetching are important performance techniques.

4. Suggest optimizations to improve performance.

- Use DNS caching/prefetching (e.g., `<link rel="dns-prefetch">`) so repeat or predicted lookups are faster.
- Use HTTP/2 or HTTP/3 and keep-alive/persistent connections to avoid repeated connection setup overhead for multiple requests.
- Employ a Content Delivery Network (CDN) to serve resources from servers geographically closer to the user, reducing network latency.
- Optimize server-side processing (caching, database indexing, efficient queries) to shorten step 4.

- Minify and compress HTML/CSS/JS, defer or async-load non-critical scripts, and optimize/lazy-load images to speed up rendering in step 6.
- Enable browser caching (via Cache-Control/ETag headers) so repeat visits skip redundant network round trips altogether.